

Topic 2: Multi-Agent Architecture — Scatter-Gather vs. Monolithic LLM Prompts

TOPIC OBJECTIVE

Monolithic LLM prompts fail on clinical charts due to attention dilution, hallucination of lab numbers, and inability to guarantee deterministic rules. AegisClinical solves this by decomposing clinical evaluation into a Directed Acyclic Graph (DAG) using LangGraph. Three domain specialists execute concurrently, accumulating findings through parallel state reducers before synchronizing at a Lead Triage Coordinator.

1. The Monolithic Prompt Fallacy: Why 'One Big Prompt' Fails

A common architectural mistake in medical AI prototypes is **Prompt Stuffing**: concatenating the patient's entire 50-page record into a single prompt and asking: 'Find all medical issues.' In production environments, this fails catastrophically for three reasons:

- **Lost in the Middle:** Attention mechanisms in transformer models prioritize the beginning and end of long prompts. Critical middle tokens—such as a baseline creatinine test from six months ago—are routinely ignored.
- **Domain Interference:** Asking an LLM to simultaneously calculate creatinine percentage changes, cross-reference drug interaction databases, and code comorbidities leads to confabulation. It mixes past lab results with present ones.
- **Lack of Deterministic Verification:** You cannot unit test a prompt. A single prompt can never provide the mathematically verifiable guarantees required for clinical decision support.

2. The Scatter-Gather (Fan-Out / Fan-In) Architecture

AegisClinical organizes its evaluation pipeline as a **Directed Acyclic Graph (DAG)** using **LangGraph**. The execution flow uses the *Scatter-Gather* pattern:

Phase	Graph Node	Operational Responsibility
1. INGESTION	parse_synthea_bundle	Strips raw FHIR JSON into structured patient demography, time-series labs, medications, and conditions.
2. SCATTER (Fan-Out)	lab_agent_node pharma_agent_node history_agent_node	Three parallel sub-agents execute simultaneously on isolated state slices. Lab calculates KDIGO AKI; Pharma checks O(1) DDI hash tables; History normalizes chronic diagnoses.

3. GATHER (Fan-In)	triage_coordinator_node	Barrier synchronization point. Aggregates all alerts, computes composite priority (HIGH/MED/LOW), orders clinical directives, and triggers local SLM summary.
-------------------------------------	-------------------------	---

3. Concurrency Without Race Conditions: The Reducer Pattern

When multiple agents execute concurrently in Python, naive dictionary updates would overwrite each other. AegisClinical prevents this using **typing.Annotated** with **operator.add** reducers in src/schemas.py:

```
class ClinicalGraphState(TypedDict):
    patient_id: str
    raw_labs: List[Dict[str, Any]]
    raw_medications: List[Dict[str, Any]]

    # Parallel reducers: operator.add appends list deltas concurrently
    lab_alerts: Annotated[List[LabAlert], operator.add]
    drug_interactions: Annotated[List[DrugInteraction], operator.add]
    chronic_conditions: Annotated[List[ChronicCondition], operator.add]
    triage_assessment: Optional[TriageAssessment]
```

How it works: When lab_agent_node returns {'lab_alerts': [Alert1]}, LangGraph does not overwrite the state. It uses operator.add(state['lab_alerts'], [Alert1]), guaranteeing zero data loss and thread-safe execution across all concurrent branches.

4. Real-World Execution Trace: Arthur Morales

Here is the exact lifecycle of Patient 01 (Arthur Morales) as it passes through the multi-agent graph:

Agent / Node	Input Consumed	Output Emitted to State
Lab Specialist	Creatinine: 1.0 & 2.4; Potassium: 5.2	LabAlert(KDIGO_AKI_STAGE_2, CRITICAL)
Pharma Specialist	Lisinopril + Furosemide + Ibuprofen	DrugInteraction(Triple Whammy, CONTRAINDICATED)
History Specialist	Condition: Essential Hypertension (I10)	ChronicCondition(I10, active)
Lead Coordinator	All alerts gathered from preceding 3 agents	Priority: HIGH (Directives: 1. Emergent ECG, 2. Stop NSAID)

KNOWLEDGE CHECK & NEXT STEP

1. Why is Scatter-Gather superior to a sequential agent chain?

Parallel agents reduce evaluation latency from ~3 seconds to ~50ms and eliminate bias contamination between specialists.

2. What prevents state collisions when three agents write to state at once?

LangGraph's `Annotated[List[T], operator.add]` reducers append state changes rather than overwriting.

3. Next Step in Topic 3: Healthcare Data Standards — Decentering FHIR R4, LOINC, and RxNorm ontologies.